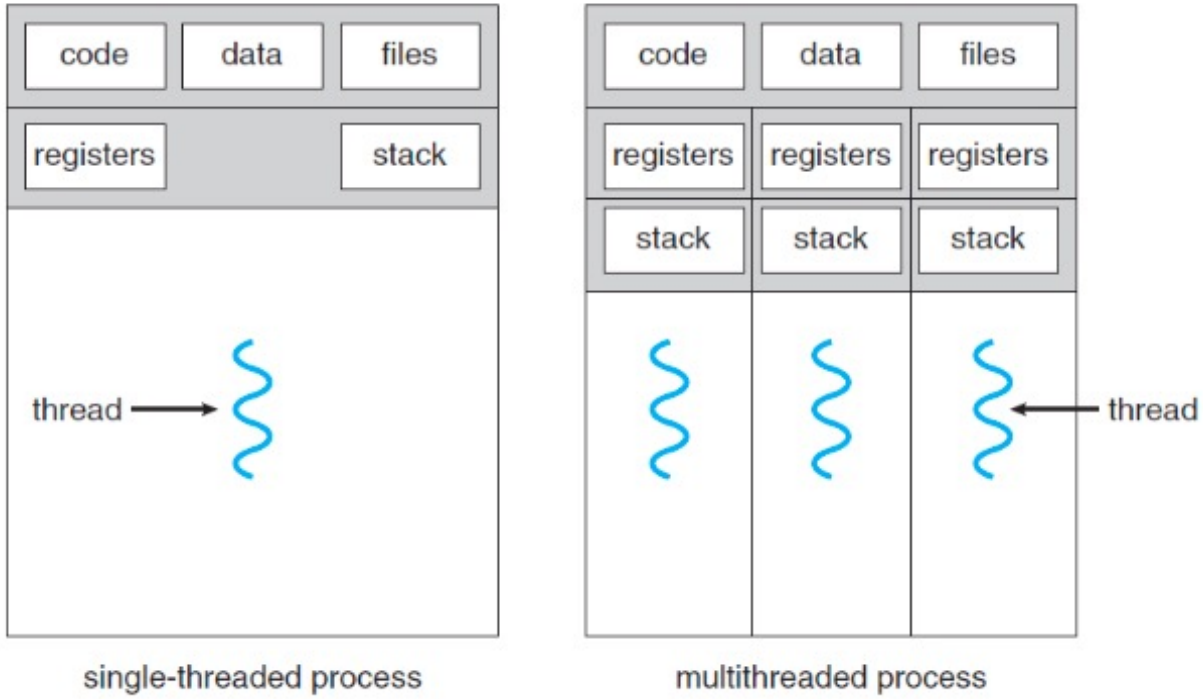
Recap: Processes & Threads

- A traditional process has a single thread.
- Multiple threads in a process can perform more than one task at a time.
- Threads in a process share code section, data section, and other OS resources, e.g., open files.
  - separate thread ID, program counter (PC), register set, and stack



single-threaded process

multithreaded process



CPU Scheduling

Ivy Peng

Associate Professor

EECS, KTH

Outline

- **Scheduling Basics**

- Scheduling Algorithms
  - FCFS
  - SJF
  - RR
  - PS
  - MQS

- Scheduling Implementation
  - Linux Kernel
  - Thread
  - Multi-processor

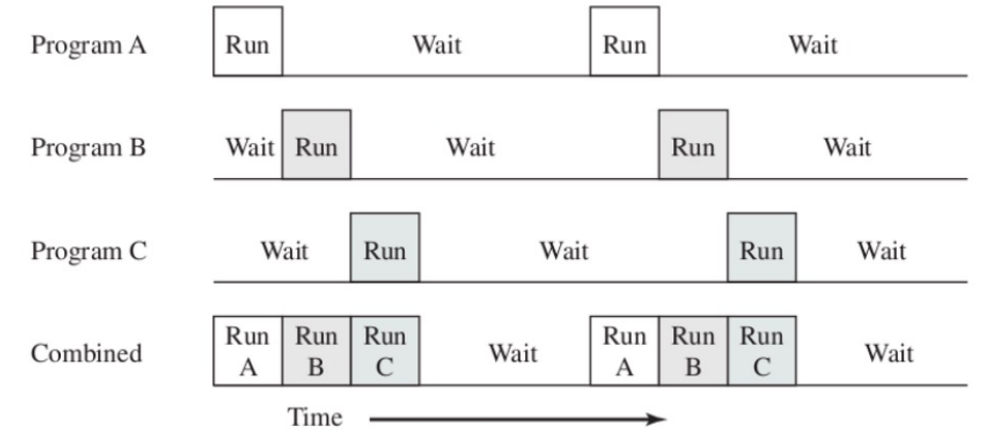
By the End of This Lecture

- Q1 The scheduling criteria, how to select a single algorithm?
- Q2 Given arrival and burst times, compute metrics under FCFS, SJF, and Round Robin.
- Q3 Convoy effect and starvation: which algorithms suffer each, and what fixes them?
- Q4 What does Linux actually run today? What changed from CFS to EEVDF?

CPU Scheduling

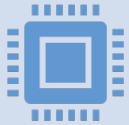
CPU scheduling is the basis of **multiprogrammed** OSs.

By switching the CPU among processes, the OS makes the computer more productive.



Courtesy: W. Stallings, Operating Systems: Internals and Design Principles, 2011

Basic Concepts



In a single-processor system, only one process can run at a time.



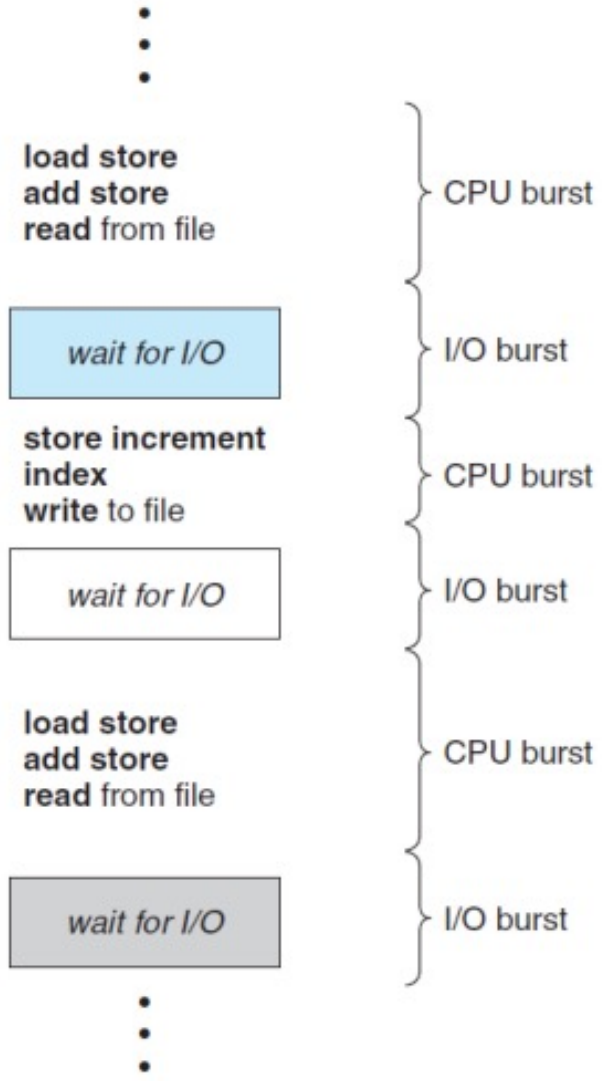
Others must wait until the CPU is free and can be rescheduled.



The objective of multiprogramming is to have some process running at all time, to maximize CPU utilization.

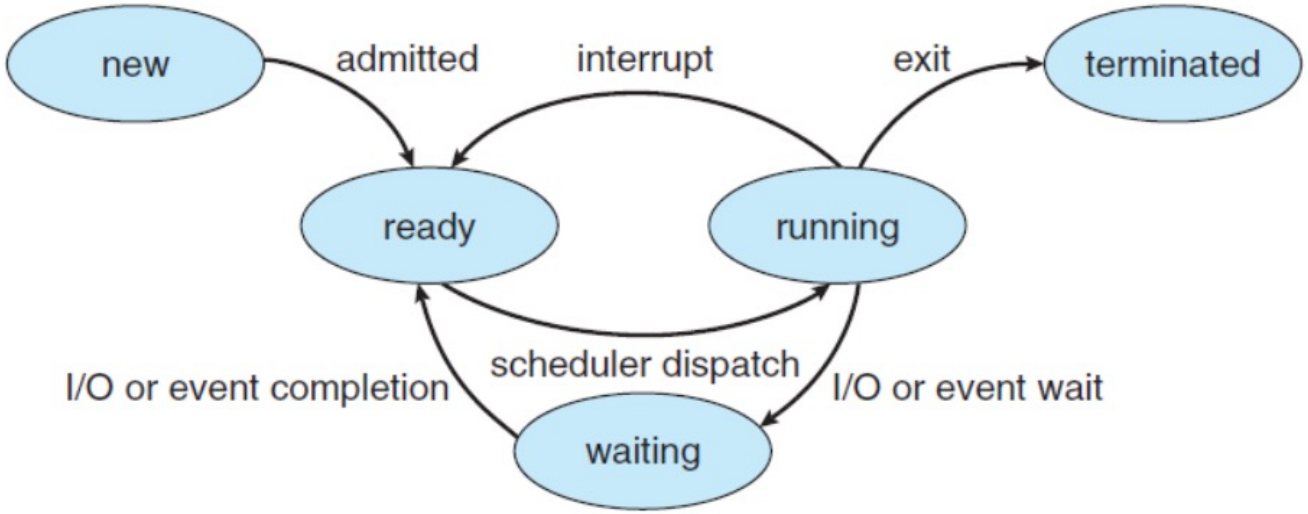
CPU I/O Burst Cycle

- Process execution consists of a cycle of CPU execution and I/O wait:
  - CPU-burst: useful computation
  - I/O-burst: CPU is idle, waiting for I/O event
- CPU burst distribution is of main concern.



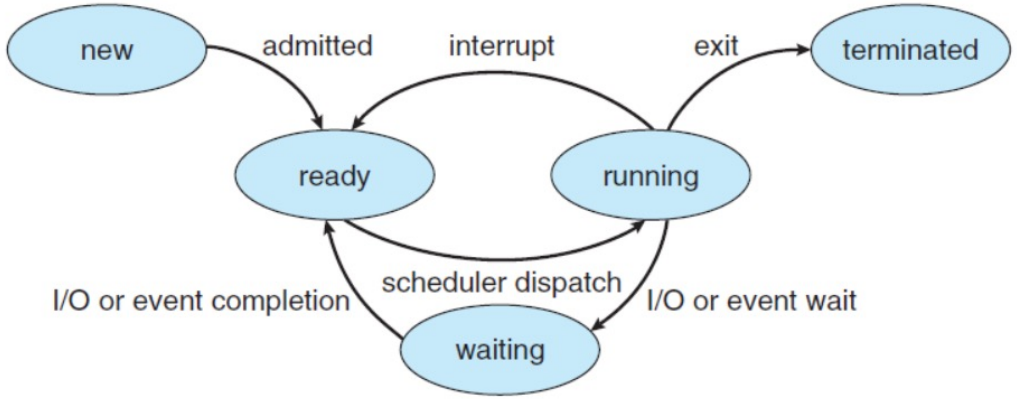
Process States (revisited)

- As a process executes, it changes state.
  - **New**: The process is being created.
  - **Ready**: The process is waiting to be assigned to a processor.
  - **Running**: Instructions are being executed.
  - **Waiting:** The process waits for an event to occur.
  - **Terminated**: The process has finished execution.



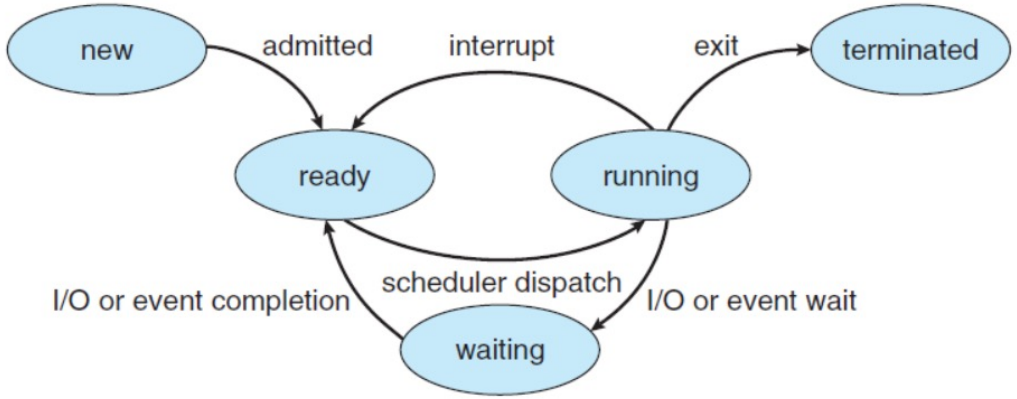
CPU Scheduler

- The CPU scheduler selects from processes in the ready queue and allocates the CPU to one of them:
  - For simplicity, we only consider **single-processor** systems for now



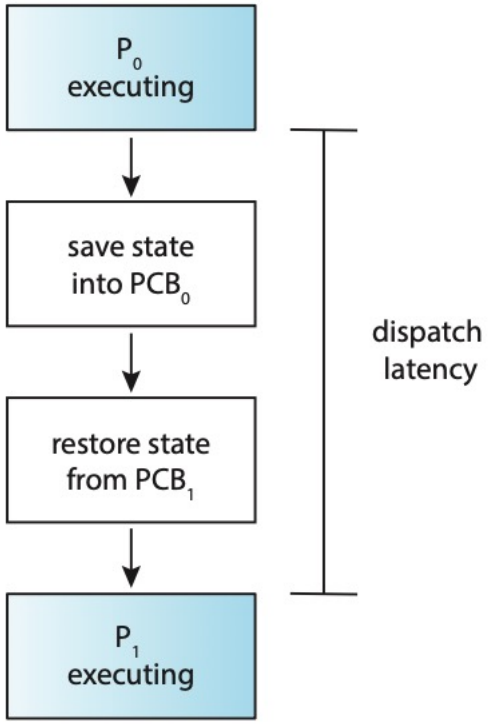
CPU Scheduler

- Scheduling choices are not needed for
  - Switches from running to waiting (e.g., I/O burst starts).
  - Terminates
- Scheduling choices are needed for the ready queue:
  - Switches from running to ready (e.g., interrupt).
  - Switches from waiting to ready (e.g., I/O burst ends)
  - And also when a new process arrives



Context Switching (Revisit)

- The scheduler makes decisions
  - What states does P0 transition
  - What states does P1 transition
- The dispatcher
  - Switch context from one process to another
  - Switch to user mode
  - Jump to resume the program from proper location (by PC)
- Dispatch Latency is critical due to the high frequency of context switch



Context Switching

- You can monitor System Level context switching via command '*vmstat*'

```
ip@ip-QEMU-Virtual-Machine:/shared$ vmstat 3 10
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 1  0      0 1458188  65588 1579192    0    0     4     4   16   23  0  0 100  0  0
 0  0      0 1458188  65588 1579192    0    0     0     0  149  193  1  0 99  0  0
 0  0      0 1458188  65588 1579192    0    0     0     1  139  175  1  0 99  0  0
 0  0      0 1458188  65588 1579192    0    0     0     0  126  162  1  0 99  0  0
 0  0      0 1458188  65588 1579192    0    0     0     0  982 1527  1  1 97  0  0
 0  0      0 1458188  65588 1579192    0    0     0     0  644  981  1  1 98  0  0
 0  0      0 1458188  65588 1579192    0    0     0     0  146  191  1  0 99  0  0
 0  0      0 1458188  65588 1579192    0    0     0     0  139  182  1  0 99  0  0
 0  0      0 1458188  65588 1579192    0    0     0     0  148  195  1  0 99  0  0
 0  0      0 1458188  65588 1574968    0    0     0     0 1429 1905 12  1 87  0  0
```

Procs
- r: The number of processes waiting for run time.
- b: The number of processes in uninterruptible sleep.

Memory
- swpd: the amount of virtual memory used.
- free: the amount of idle memory.
- buff: the amount of memory used as buffers.
- cache: the amount of memory used as cache.
- inact: the amount of inactive memory. (-a option)
- active: the amount of active memory. (-a option)

Swap
- si: Amount of memory swapped in from disk (/s).
- so: Amount of memory swapped to disk (/s).

IO
- bi: Blocks received from a block device (blocks/s).
- bo: Blocks sent to a block device (blocks/s).

System
- in: The number of interrupts per second, including the clock.
- cs: The number of context switches per second.

CPU

These are percentages of total CPU time.
- us: Time spent running non-kernel code. (user time, including nice time)
- sy: Time spent running kernel code. (system time)
- id: Time spent idle. Prior to Linux 2.5.41, this includes IO-wait time.
- wa: Time spent waiting for IO. Prior to Linux 2.5.41, included in idle.
- st: Time stolen from a virtual machine. Prior to Linux 2.6.11, unknown.

Context Switching

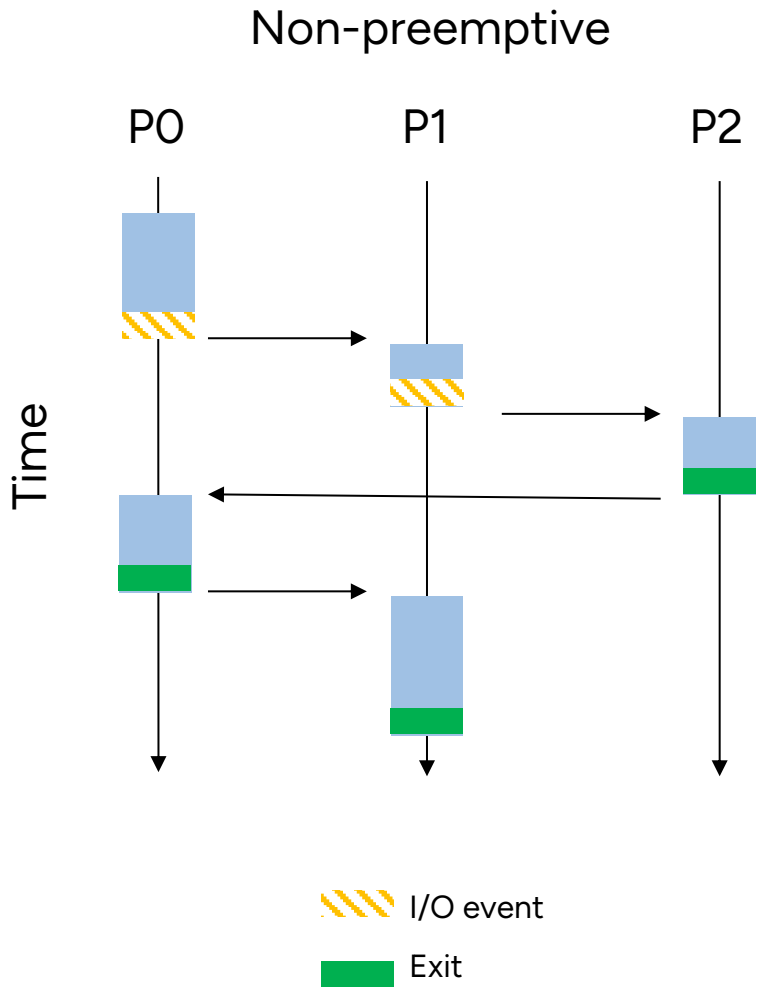
- You can also check Process Level context switching via querying '*/proc/PID/status*'

```
ip@ip-QEMU-Virtual-Machine:/shared$ cat /proc/511/status
Name:   scsi_eh_0
Umask:  0000
State:  S (sleeping)
Tgid:   511
Ngid:   0
Pid:    511
PPid:   2
TracerPid:      0
Uid:    0       0       0       0
Gid:    0       0       0       0
FDSize: 64
Groups:
NStgid: 511
NSpid:  511
NSpgid: 0
NSsid:  0
Kthread:        1
Threads:        1
SigQ:   0/15156
SigPnd: 0000000000000000
ShdPnd: 0000000000000000
SigBlk: 0000000000000000
SigIgn: ffffffffffffffff
SigCgt: 0000000000000000
CapInh: 0000000000000000
CapPrm: 000001ffffffffff
CapEff: 000001ffffffffff
CapBnd: 000001ffffffffff
CapAmb: 0000000000000000
NoNewPrivs:     0
Seccomp:        0
Seccomp_filters:        0
Speculation_Store_Bypass:       vulnerable
SpeculationIndirectBranch:      unknown
Cpus_allowed:   f
Cpus_allowed_list:      0-3
Mems_allowed:   00000000,00000001
Mems_allowed_list:      0
voluntary_ctxt_switches:        2
nonvoluntary_ctxt_switches:     0
```

Nonpreemptive vs. Preemptive Scheduling

- Nonpreemptive scheduling: when a CPU is allocated to a process P0, P0 keeps running on the CPU until:
  - P0 terminates
  - P0 switches from running to waiting

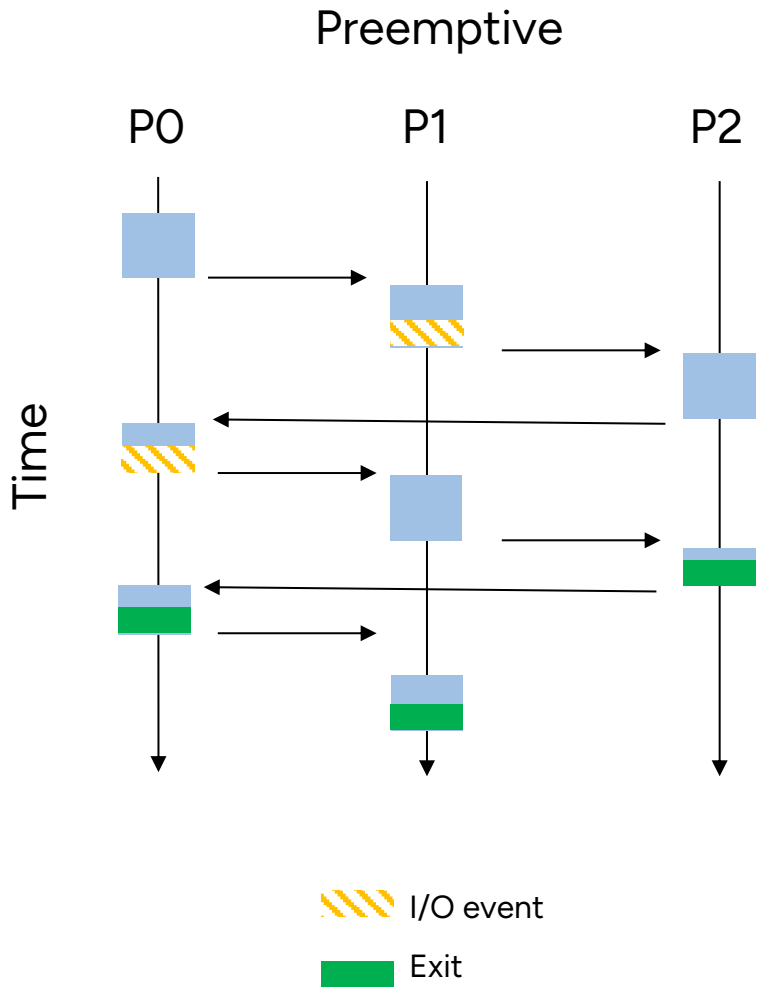




Nonpreemptive vs. Preemptive Scheduling

- Preemptive scheduling:
  - a process switches from the running state to the ready state (e.g., a timer interrupt)
  - a process switches from the waiting state to the ready state (e.g., an I/O event finishes)
- All modern operating systems use preemptive scheduling





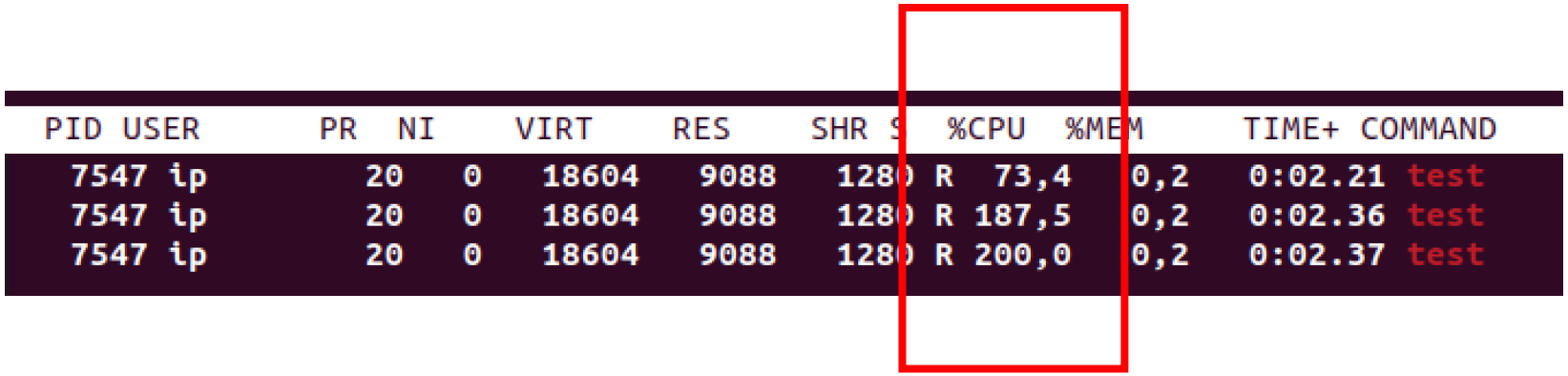
Scheduling Criteria

Different CPU scheduling algorithms have different properties.

- **CPU utilization**: keep the CPU as busy as possible.
- **Throughput**: # of completed processes per time unit.
  - =#/Makespan
- **Turnaround time**: amount of time to execute a particular process.
- **Waiting time**: the amount of time a process has been waiting in the ready queue.
- **Response time**: the amount of time it takes from when a request was submitted until the first response is produced.

Scheduling Criteria – CPU Utilization

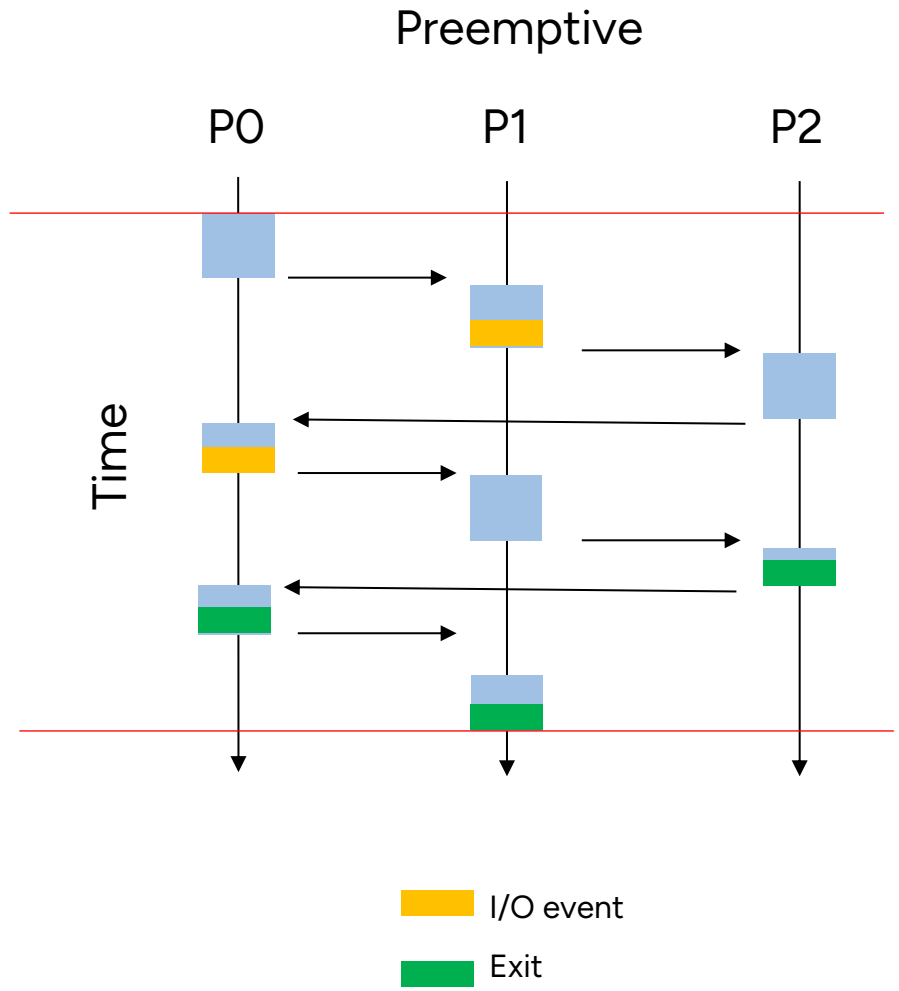
- Scheduling Goal: keep the CPU as busy as possible.
- CPU utilization = the % of time that CPU is busy
- On a single core, CPU utilization can range 0 – 100
- On multi-core systems, you might see CPU Util > 100%, because a process can leverage multiple CPUs



PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
7547	ip	20	0	18604	9088	128	R	73,4	0,2	0:02.21	test
7547	ip	20	0	18604	9088	128	R	187,5	0,2	0:02.36	test
7547	ip	20	0	18604	9088	128	R	200,0	0,2	0:02.37	test

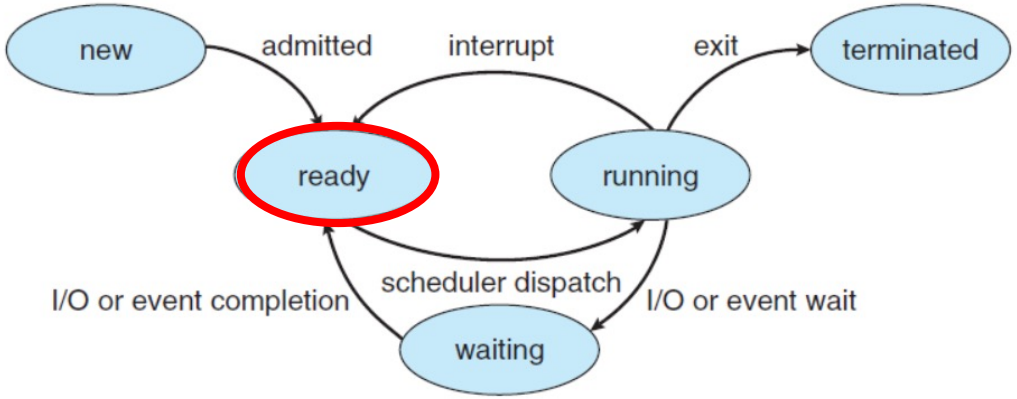
Scheduling Criteria – Throughput

- Scheduling Goal: from the system perspective, it wants to have more completed processes per time unit.
- Throughput = #completed processes / Time
  - more specifically, time = makespan = the beginning of the 1st job till the completion of the last job
- It depends on the duration of each process:
  - Long processes will reduce throughput
  - Short processes will increase throughput



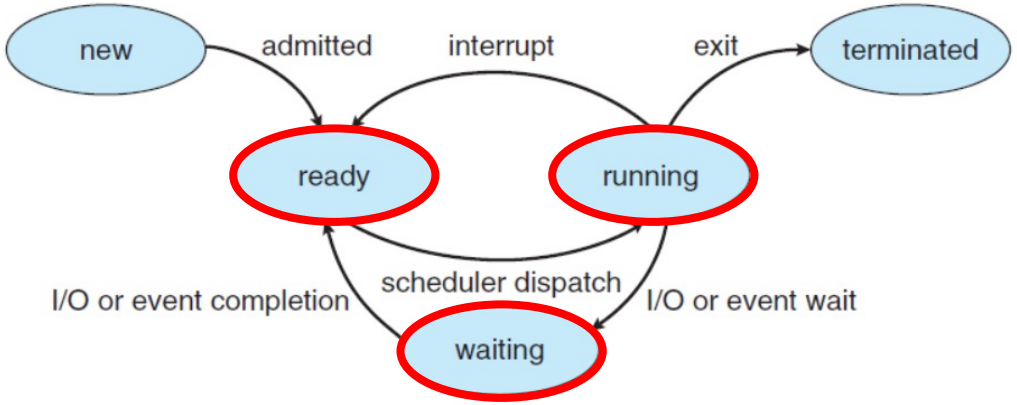
Scheduling Criteria – Waiting time

- Scheduling Goal: for each process perspective, how long it spends in waiting for execution, i.e., in the Ready queue.
- Waiting Time = the time spent in the ready queue
- Preemptive v.s Nonpreemptive scheduling impact?



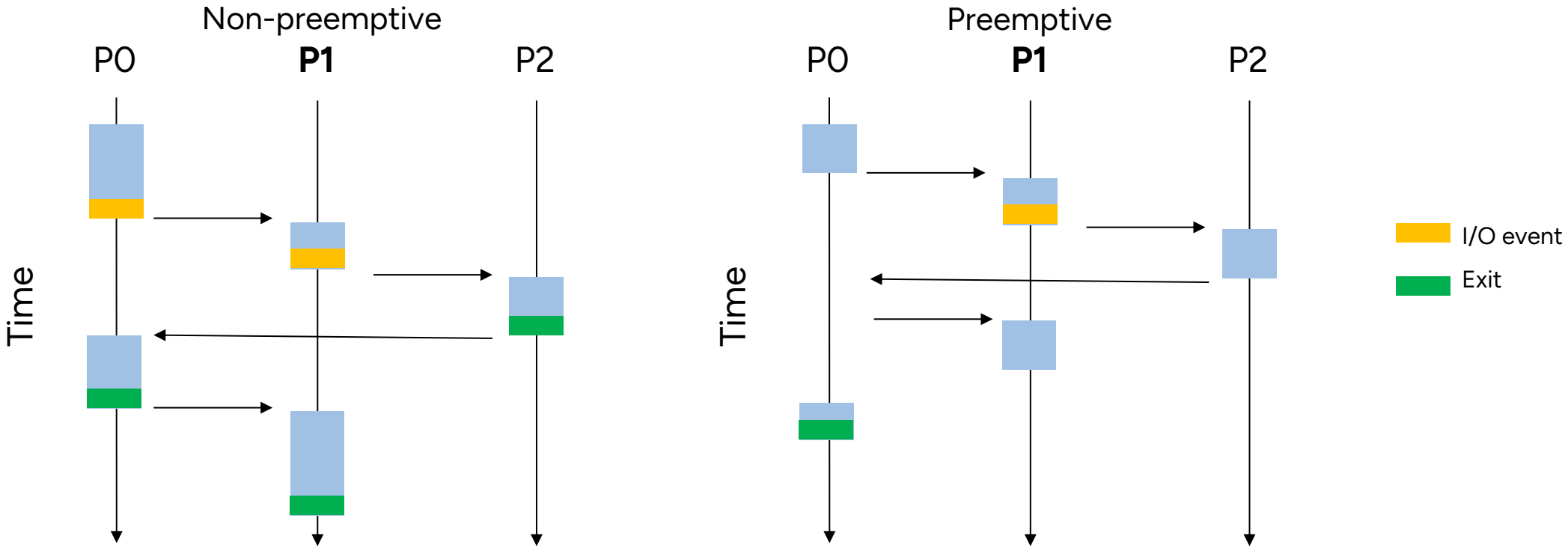
Scheduling Criteria – Turnaround Time

- Scheduling Goal: for each process perspective, how long it takes to execute that process.
- The interval from the time of submission of a process to the time of completion is the turnaround time.
- Turnaround time = $T_{completion} - T_{arrival}$
- The longer a process spends in **ready**, **running, and waiting queues**, the longer is its turnover time



Scheduling Criteria – Response Time

- Scheduling Goal: faster to give out the first response (could be partial output)
- The interval from the time of submission of a process to the time the first response is produced.
- Important for interactive systems
  - Rate-Monotonic Scheduling is often used for real-time system requires high responsiveness





Outline

- Scheduling Basics

- **Scheduling Algorithms**
  - **FCFS**
  - SJF
  - RR
  - PS
  - MQS

- Scheduling Implementation
  - Linux Kernel
  - Thread
  - Multi-processor

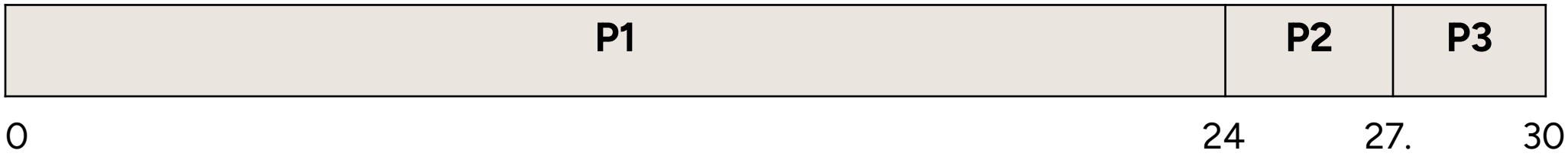
FCFS Scheduling

- First-come First-serve: the process that arrives earlier gets scheduled to run earlier
- Suppose that the processes arrive in the order: P1, P2, P3, at timestamp 0, and Burst Time is given below

Process	Burst Time
P1	24
P2	3
P3	3

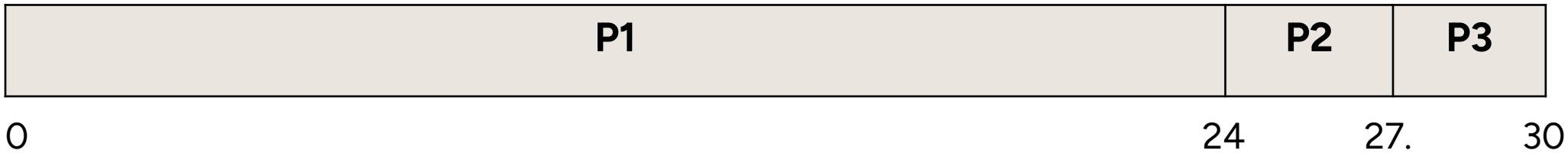
Questions: Draw a Gantt chart to illustrate the schedule.

FCFS Scheduling



What is Average Waiting Time?

FCFS Scheduling

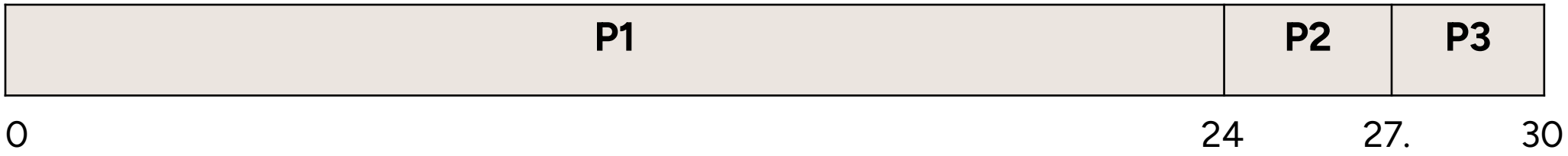


What is Average Waiting Time?

- Waiting Time for P1 = 0; P2 = 24; P3 = 27
- Average waiting time:
  - (0+24+27) / 3 = 17

FCFS Scheduling – Turnaround Time

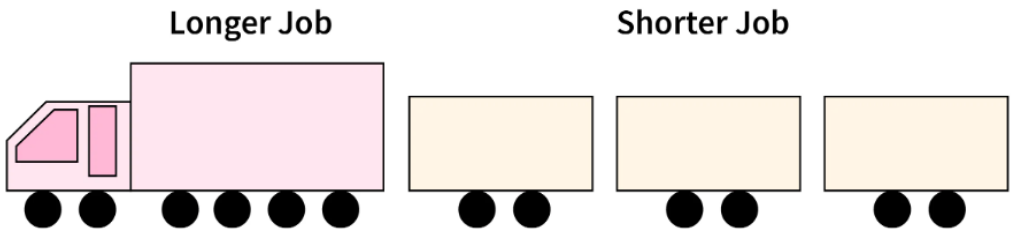
- Turnaround time = T_process_completion – T_process_arrival
- P1, P2, P3 arrive at time 0.
  - P1: (24-0)=24; P2: (27 – 0) = 27; P3: (30-0)=30;
  - Average Turnaround = 27
- What if I the arrival time changes to: P1 = 0; P2 = 10; P3 = 12?



Convoy Effect

- **Convoy effect**: all the other processes wait for the one big process to get off the CPU.

- **FCFS scheduling is non-preemptive**: the process keeps the CPU until it releases the CPU
  - If an earlier process is long, all later processes have to wait



FCFS Scheduling

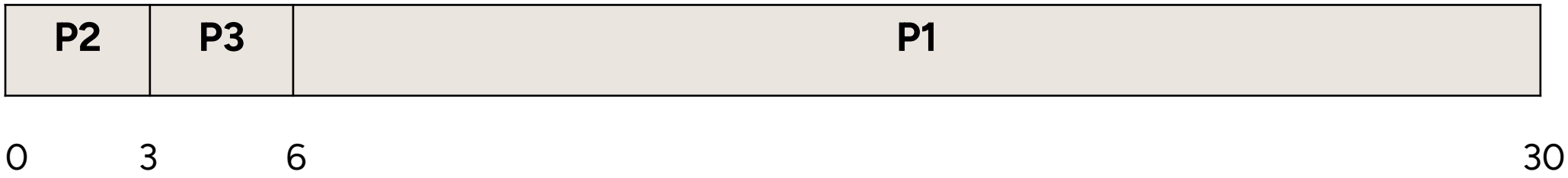
- Suppose that the processes arrive in the **order: P2, P3, P1** (still arrival time is 0)

Process	Burst Time
P2	3
P3	3
P1	24

Draw a Gantt chart that illustrates the schedule.

FCFS Scheduling

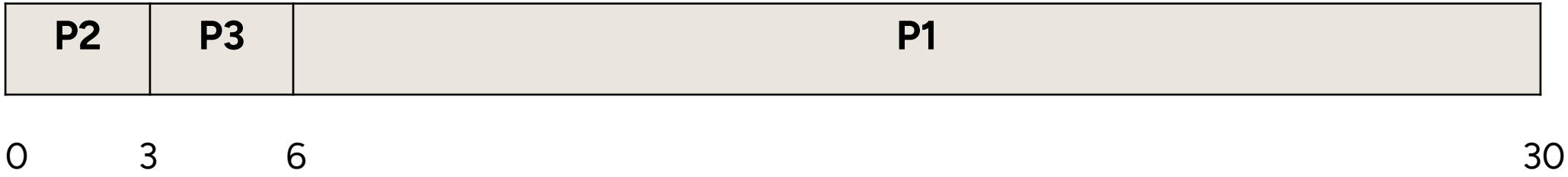
- Suppose that the processes arrive in the **order: P2, P3, P1**



- Waiting time for P2 = 0; P3 = 3; P1 = 6;
- Average Waiting Time:
  - (6+0+3) / 3 = 3
- Much better than the previous case.

FCFS Scheduling – Turnaround Time

- Turnaround time = Tprocess_completion – Tprocess_arrival
  - P1: (30-0)=30;
  - P2: (3 – 0) = 3;
  - P3: (6-0)=6;
- Average Turnaround = 13



Outline

- Scheduling Basics

- **Scheduling Algorithms**
  - FCFS
  - **SJF**
  - RR
  - PS
  - MQS

- Scheduling Implementation
  - Linux Kernel
  - Thread
  - Multi-processor

SJF Scheduling



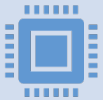
Associate each process with **the length of its next CPU burst**.



Use these lengths to schedule the process with the shortest time.



SJF is optimal: it gives the minimum average waiting time for a given set of processes.



Challenge: ?

SJF Scheduling

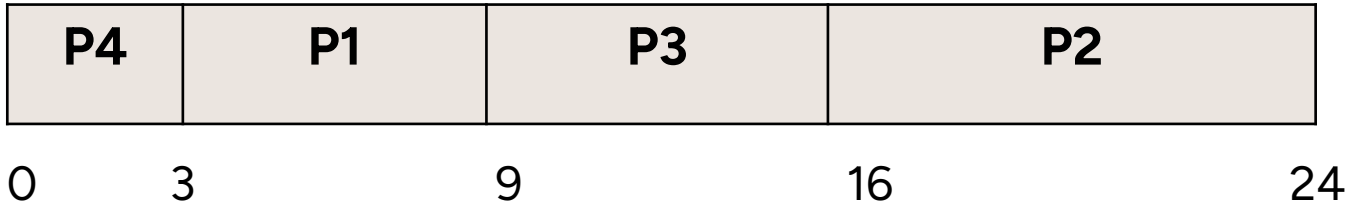
- Select the processes according to their burst time (from shorter to longer).
- Suppose that the processes arrive at the same time, and Burst Time is given below

Process	Burst Time
P1	6
P2	8
P3	7
P4	3

Draw a Gantt chart that illustrates the schedule.

SJF Scheduling

- **Draw a Gantt chart that illustrates the schedule.**

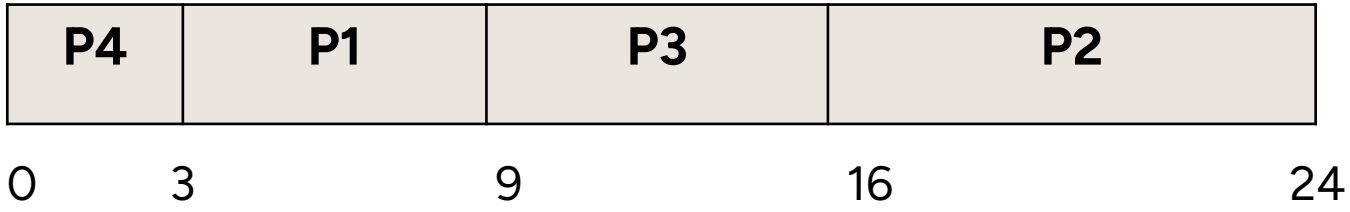


Process	Burst
P1	6
P2	8
P3	7
P4	3

- **Waiting time** for P1 = 3; P2 = 16; P3 = 9, P4 = 0
- Average waiting time:
  - (3+16+9+0) /4 = 7

SJF Scheduling

- **Draw a Gantt chart that illustrates the schedule.**



Process	Burst
P1	6
P2	8
P3	7
P4	3

- **Turnaround time** for P1 = 9; P2 = 24; P3 = 16, P4 = 3
- Average Turnaround Time:
  - (9+24+16+3) /4 = 13

Challenge: Burst Time Prediction

- In reality, you cannot know a job/process's burst time in advance
  - The total job length may be unknow
  - The occurrence of the next I/O event may be dynamic
- SFJ needs accurate prediction to work efficiently

- The most commonly used prediction method is **exponential average**:

$$\tau_{n+1} = \alpha \cdot t_n + (1-\alpha) \cdot \tau_n$$

Burst Time Prediction

- Prediction via **exponential average**:

$$\tau_{n+1} = \alpha \cdot t_n + (1-\alpha) \cdot \tau_n$$

- τ_{n+1} is the predicted time for the next burst
- τ_{n+1} is the predicted time for the current burst. Start with a random guess at time 0.
- t_n is the actual measured time for the current burst
- α is the weight factor, $0 \leq \alpha \leq 1$

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\alpha t_{n-1} + \cdots + (1-\alpha)^j \alpha t_{n-j} + \cdots + (1-\alpha)^{n+1} \tau_0.$$

Question: if you want to weigh your prediction more on the recent records, how will you adapt?

SJF Scheduling

Given the following CPU bursts of two processes.

Process 0

α =0.5	i=0	i=1	i=2	i=3	i=4	i=5	i=6
Guess τ_i	10						
CPU Burst (t_i)	6	8	10	6	10	9	10

Process 1

α =0.5	i=0	i=1	i=2	i=3	i=4	i=5	i=6
Guess τ_i	8						
CPU Burst (t_i)	8	4	4	5	4	6	4

SJF Scheduling

Given the following CPU bursts of two processes.

Process 0

α =0.5	i=0	i=1	i=2	i=3	i=4	i=5	i=6
Guess τ_i	10	8	8	9	7.5	8.75	8.87
CPU Burst (t_i)	6	8	10	6	10	9	10

Process 1

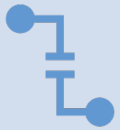
α =0.5	i=0	i=1	i=2	i=3	i=4	i=5	i=6
Guess τ_i	8	8	6	5	5	4.5	5.25
CPU Burst (t_i)	8	4	4	5	4	6	4

SJF Scheduling

- Advantages:
  - Short waiting time
- Disadvantage:
  - prediction, complex,
  - **Starvation: long jobs may be kept in the queue**



Preemptive SJF



The SJF algorithm can be either preemptive or non-preemptive.



The preemptive version called **shortest-remaining-time-first (SRTF)**

Preemptive SRTF

- Select the processes with the shortest remaining time to run.
- Now, we add the concepts of **varying arrival times and preemption** to the analysis.
- Suppose that the processes arrival time and Burst Time is given below

Process	Arrival Time	Burst Time	Remaining Time
P1	0	8	{P1: 8}
P2	1	4	
P3	2	9	
P4	3	5	

Preemptive SRTF

- Select the processes with the shortest remaining time to run.
- Now, we add the concepts of varying arrival times and preemption to the analysis.
- Suppose that the processes arrival time and Burst Time is given below

Process	Arrival Time	Burst Time	Remaining Time
P1	0	8	{P1: 8}
P2	1	4	{P1: 7, P2:4}
P3	2	9	
P4	3	5	

Draw a Gantt chart that illustrates the schedule.

Preemptive SRTF

- Select the processes with the shortest remaining time to run.
- Now, we add the concepts of varying arrival times and preemption to the analysis.
- Suppose that the processes arrival time and Burst Time is given below

Process	Arrival Time	Burst Time	Remaining Time
P1	0	8	{P1: 8}
P2	1	4	{P1: 7, P2:4}
P3	2	9	{P1: 7, P2:3, P3:9}
P4	3	5	{P1: 7, P2:2, P3:9, P4:5}

Draw a Gantt chart that illustrates the schedule.

Preemptive SRTF

- **Draw a Gantt chart that illustrates the schedule.**



- P1 is preemptive when P2 arrives.
- Average waiting time:
  - ((10−1)+(1−1)+(17−2)+(5−3)) / 4 = 26/4 = 6.5

Round Robin

- Each process gets a small unit of CPU time slice (Time **Quantum** q), usually 10-100 milliseconds.
- After a time quantum has elapsed, the process is preempted and added to the end of the ready queue.
- If there are N processes in the ready queue, then:
  - each process gets at most 1/N of the CPU time.
  - No process waits more than (n − 1)q time units – no **starvation**

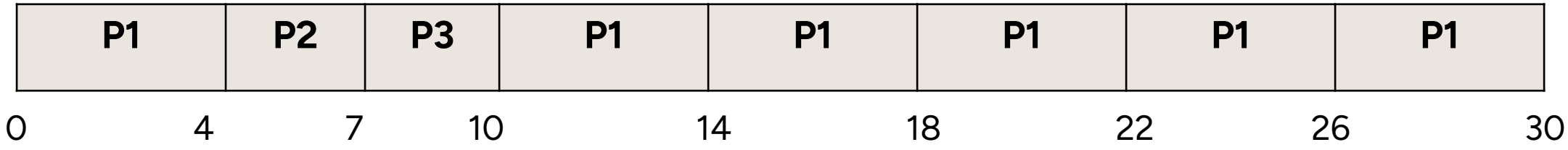
- Implementation: the ready queue is implemented as a circular buffer

Round Robin – Case 1

- RR runs similar to FCFS, with addition of pre-empting a process when a Time Quantum finishes
- Assume time quantum q = 4

Process	Burst Time
P1	24
P2	3
P3	3

Draw a Gantt chart that illustrates the schedule.



- Average Waiting Time: ((10−4)+4+7) / 3 = 5.66

Round Robin – Case 2

- RR runs similar to FCFS, with addition of pre-empting a process when a Time Quantum finishes
- Assume time quantum q = 4

Process	Burst Time
P1	8
P2	4
P3	9
P4	5

Draw a Gantt chart that illustrates the schedule.



Round Robin – Case 3

- RR runs similar to FCFS, with addition of pre-empting a process when a Time Quantum finishes
- Assume time quantum q = 4

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

Draw a Gantt chart that illustrates the schedule.



Round Robin - Time Quantum q

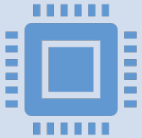
- The timer interrupts every quantum q to schedule the next process.
  - The more context switches in the completion of a process, the higher overhead
- Performance
  - q large ⇒ approximate FIFO
  - q small ⇒ q must be large with respect to the context switch; otherwise, overhead is too high.
- Choose the time quantum q such that q is longer than 80% of the CPU bursts.

- What is the impact on Average Waiting Time, when quantum q is reduced?

Priority Scheduling



A priority number (integer) is associated with each process.



The CPU is allocated to the process with the highest priority.

Smallest integer = Highest priority

Preemptive and non-preemptive



SJF is priority scheduling where priority is the inverse of the predicted next CPU burst time.

Priority Scheduling - Example

- Each process now is a given a priority
- the lower value, the higher priority in this example

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

Draw a Gantt chart that illustrates the schedule.

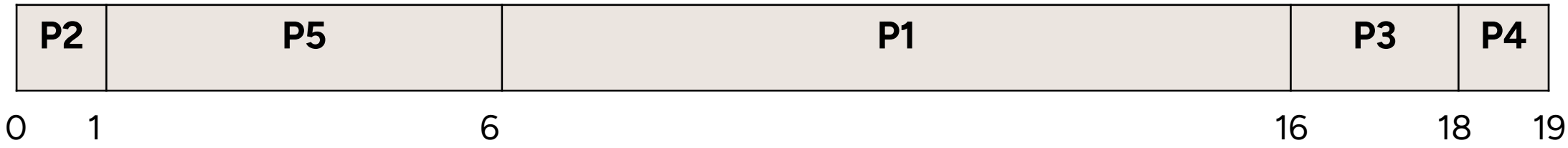


Priority Scheduling - Example

- Each process now is a given a priority
- the lower value, the higher priority in this example

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

Draw a Gantt chart that illustrates the schedule.



Average waiting time: (0+1+6+16+18)/ 5 = 8.2

Priority Scheduling

- Problem: **starvation** - low priority processes may never execute
- Solution: **aging** - as time progresses, increase the priority of the process

Process	Burst Time	Priority (t0)	Priority (t1)	Priority (t2)
P1	10	3		
P2	1	1		
P3	2	4		
P4	1	5		
P5	5	2		

Priority Inversion

- In Priority Scheduling, a running process P might be preempted when a process Q with higher priority arrives
  - Assume priority: P < Q < R
- At first, P is running
- When R arrives, R requires a resource S that is currently held by P
  - R is not runnable because it needs to wait for resource S, so P is running
- Then, Q arrives. Q does not need S, Q is runnable. So the scheduler will preempt P (but still holds S) and schedules Q to run immediately. We get:
  - Arrival: P < R < Q
  - Priority: P < Q < R
- The problem: now R is waiting while Q is running, which has lower priority and arrives later.

- Solution: priority inheritance

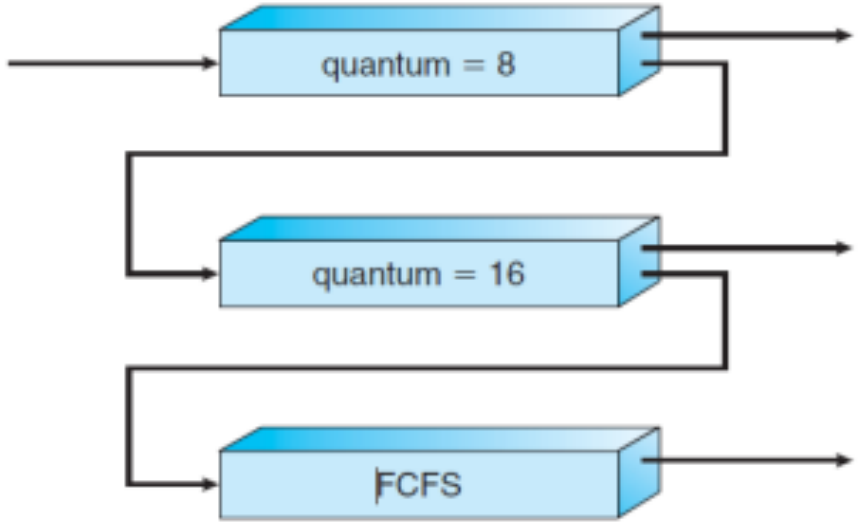
Priority Inheritance Protocol

- All processes that are holding resources needed by a higher-priority process temporarily **inherits** the priority of the blocked high-priority process

- Changes to the scenario in the previous slide:
  - When R (highest priority) arrives and tries to acquire S. S is held by P.
  - Because R is blocked on P, P inherits R's priority temporarily
  - Q arrives. P now cannot be preempted.
  - After that?

Multilevel Queue Scheduling

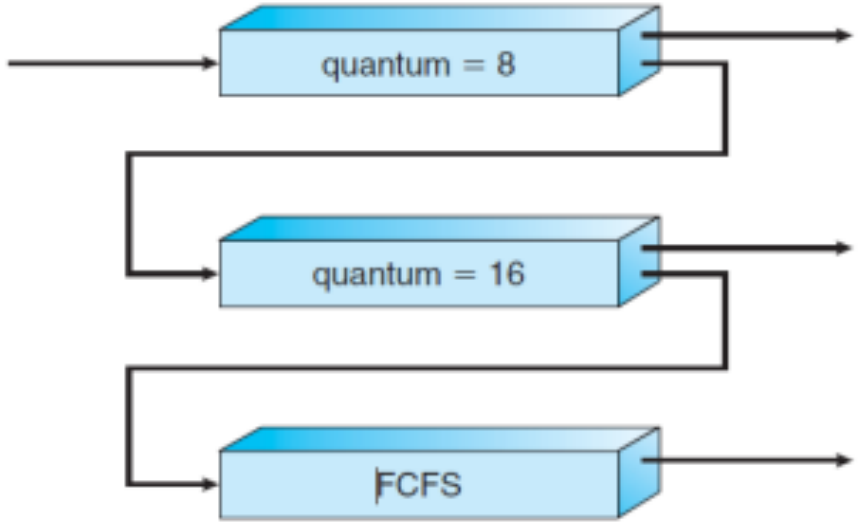
- The ready queue consists of multiple queues with different queueing policies.



- For example, three queues:
  1. Q0: RR with time quantum 8 milliseconds
  2. Q1: RR time quantum 16 milliseconds
  3. Q2: FCFS

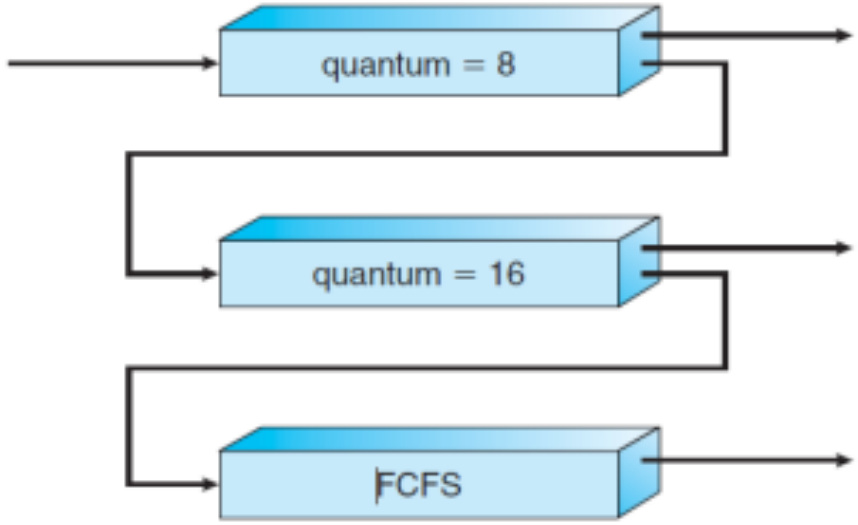
Multilevel Queue Scheduling

- For example, three queues:
  1. Q0: RR with time quantum 8 milliseconds
  2. Q1: RR time quantum 16 milliseconds
  3. Q2: FCFS
- A new job enters queue Q0, which is served RR:
  - When it gains CPU, the job receives 8 milliseconds.
  - If it does not finish in 8 milliseconds, the job is moved to queue Q1.
- At Q1, the job is again served RR and receives 16 additional milliseconds.
  - If it still does not complete, it is preempted and moved to queue Q2.



Multilevel Queue Scheduling

- In general, multilevel queue scheduler defined by the following parameters:
  - Number of queues
  - Scheduling algorithms for each queue
  - Scheduling among the queues
  - Method used to determine which queue a process will enter



Choosing a Scheduling Algorithm

Algorithm	Preemptive?	Needs burst estimate?	Starvation?	Best for	Cost / caveat
FCFS	No	No	No	simplicity	convoy effect
SJF	No	Yes	Yes	min avg waiting	prediction
SRTF	Yes	Yes	Yes	min avg waiting with arrivals	prediction
RR	Yes	No	No	response time	quantum tuning
Priority	either	No	Yes	deadlines	needs aging
MLQ / MLFQ	Yes	No	No	mixed workloads	most tuning parameters

- **SJF** assumes you know the burst times (future)
- **MLFQ approximates SJF** by learning burst behaviour through feedback.

Issues in Scheduling: Convoy & Starvation

Convoy is a waiting problem where short jobs pile up behind one big one

Starvation is a fairness problem: the victim never reaches the CPU at all

Every algorithm after FCFS exists to fix one of two failures

Issues	Example	Cause	Fix
Convoy effect	FCFS	non-preemptive: one long job at the head blocks all shorter jobs behind it	preemption, e.g., RR short-job ordering, e.g., SJF
Starvation	SJF SRTF Priority	a long or low-priority job is overtaken by shorter / higher-priority arrivals	aging rotation, e.g., RR



Outline

- Scheduling Basics

- Scheduling Algorithms
  - FCFS
  - SJF
  - RR
  - PS
  - MQS

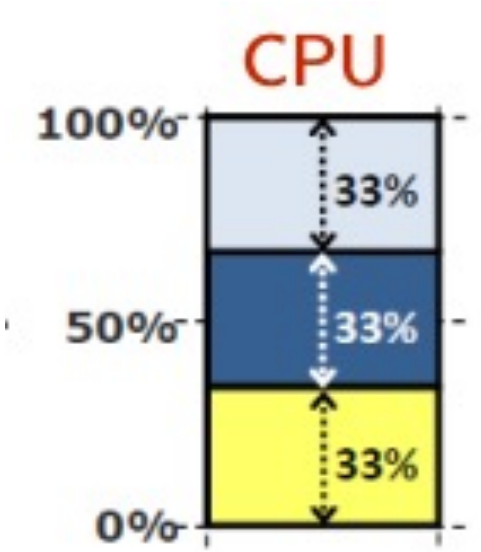
- **Scheduling Implementation**
  - **Linux Kernel**
  - **Thread**
  - **Multi-processor**

Linux Scheduling: CFS → EEVDF

- Linux used the Completely Fair Scheduler (**CFS**) from 2007 to 2023.
- Since kernel 6.6, the default scheduling changes to Earliest Eligible Virtual Deadline First (**EEVDF**).

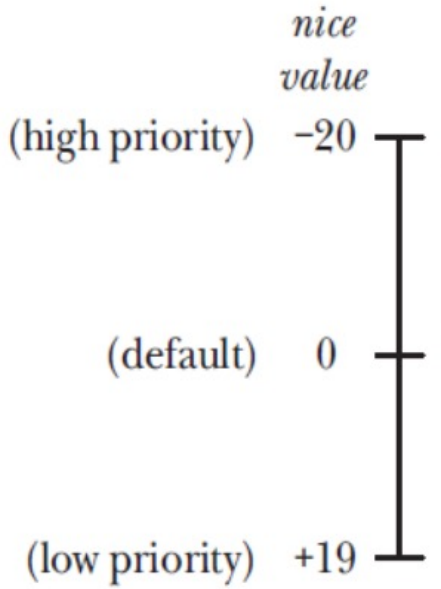
- Both CFS and EEVDF aim to give each runnable task a **fair share** of the CPU, weighted by its **nice** value.
  - So the nice value sets each task's weight

- CPU time is sliced so that each process could get their share of CPU utilization ‘fairly’
  - E.g., N users want to share a CPU, each 1/N of the shared resource.



Linux Scheduling: CFS and *Nice*

- CFS is generalized into *max-min fairness*, by allowing a *nice* value that can be set by a user
- If a user is nice, e.g., it wants less than its fair share by increasing its *nice* value
- If a user needs more priority, e.g., it wants less than its fair share by decreasing its *nice* value



Manipulate the `nice` Value

- nice() increments a process's nice value by inc and returns the newly updated value.
- Only processes owned by root may provide a negative value for inc.
- The getpriority() and setpriority() system calls allow a process to retrieve and change its own nice value or that of another process.

```
#include <unistd.h>

int nice(int inc);
```

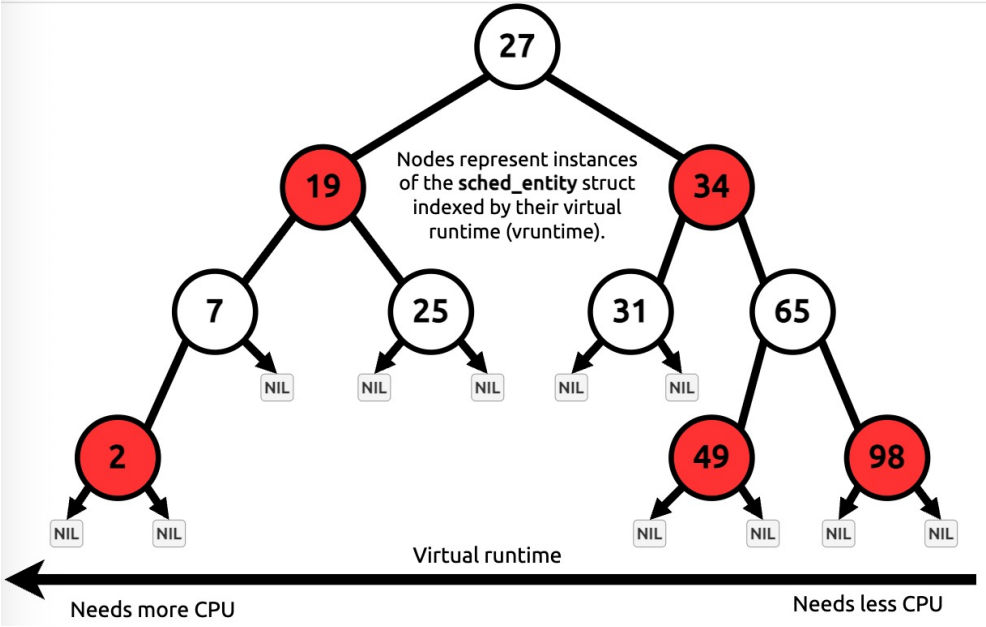
```
#include <sys/resource.h>

int getpriority(int which, id_t who);
int setpriority(int which, id_t who, int prio);
```

Linux CFS Implementation

- Linux maintains a red-black tree of sched_entity nodes for implementing CFS scheduling
  - Each sched_entity node points to a *task_struct* data structure, i.e., Linux kernel's representation of processes
  - Each node keeps a virtual clock of its runtime (vruntime).
  - Pick the next task to run = the leftmost leaf

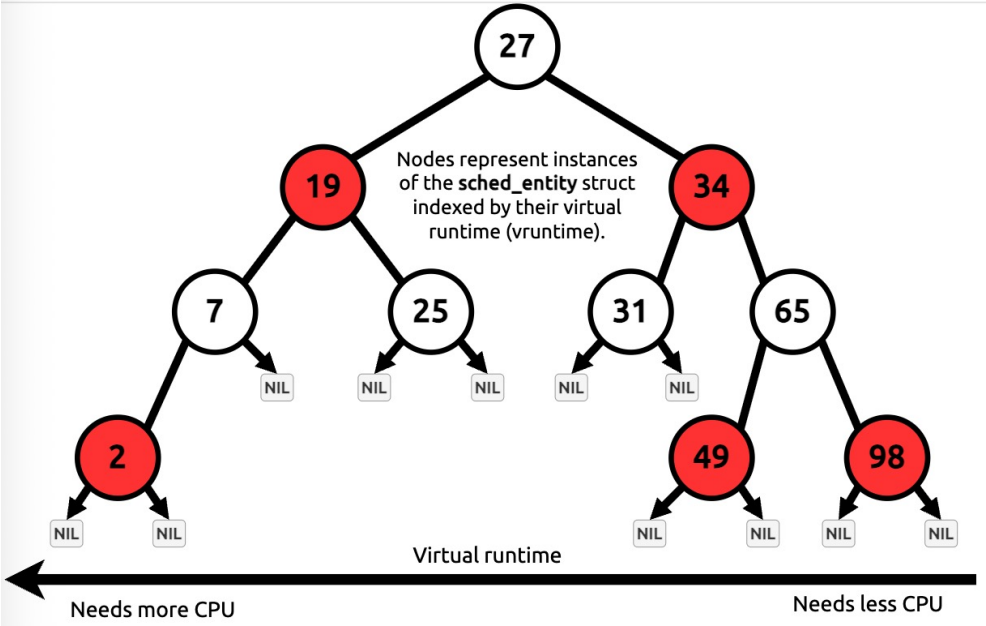
- Time quantum q (the time slice) is calculated based on the nice value
  - Tasks with lower nice values receive a higher proportion of CPU time than tasks with higher nice values
  - The nicer a task is, the later a task finishes
  - After its time quantum q elapses, q is added to the virtual clock, and the node is inserted again to the tree.



https://documentation.ubuntu.com/real-time/latest/explanation/schedulers/
https://elixir.bootlin.com/linux/v6.12.41/source/include/linux/sched.h

Linux EEVDF Implementation

- Still keeps a per-task `vruntime` (nice-weighted) in a red-black tree
- Differ from CFS in how EEVDF picks the next task to improve a task's requested **latency**
- A task is Eligible if lag ≥ 0 (it has not yet had its fair share)
- Each task has gets a virtual deadline.
- Selection[1]: among eligible tasks, pick the earliest virtual deadline

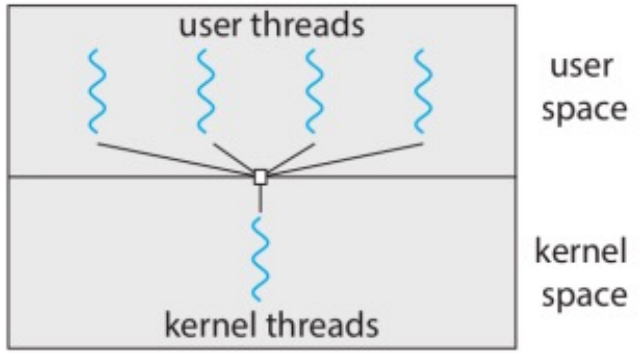


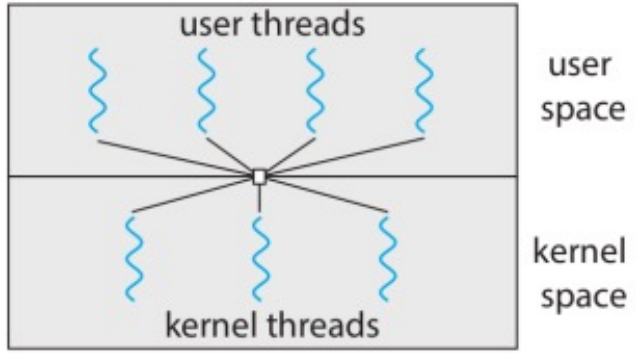
https://documentation.ubuntu.com/real-time/latest/explanation/schedulers/
https://elixir.bootlin.com/linux/v6.12.41/source/include/linux/sched.h

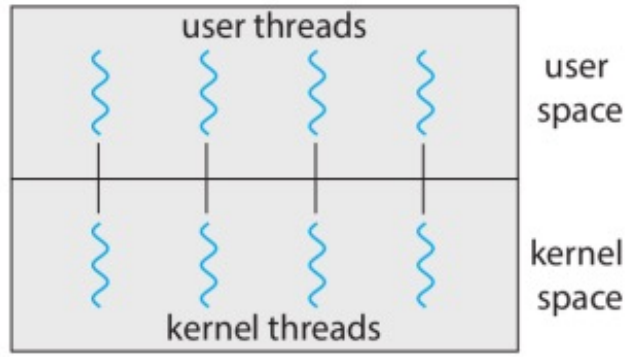
[1] https://www.kernel.org/doc/Documentation/scheduler/sched-eevdf.rst

Thread Scheduling - Contention Scope

- Distinction between user-level and kernel-level threads.
  - The OS has no knowledge of user-level threads
  - Kernel-level threads are being scheduled by the OS
  - To run on a CPU, user-level threads must be mapped to an associated kernel-level thread







Thread Scheduling - Contention Scope

- Process-Contention Scope (PCS) – user-level threads
  - In many-to-one and many-to-many models.
  - Scheduling competition is within a process.
- System-Contention Scope (SCS) – kernel-level threads
  - In a one-to-one model.
  - Scheduling competition among all threads in the system

Pthread Scheduling API

- API allows specifying either PCS or SCS during thread creation.
  - PTHREAD_SCOPE_PROCESS schedules threads using PCS scheduling.
  - PTHREAD_SCOPE_SYSTEM schedules threads using SCS scheduling.
- pthread_attr_setscope and pthread_attr_getscope set/get contention scope attribute in thread attributes object.

```
#include <pthread.h>

int pthread_attr_setscope(pthread_attr_t *attr, int scope);
int pthread_attr_getscope(const pthread_attr_t *restrict attr, int *restrict scope);
```

Linux only supports PTHREAD_SCOPE_SYSTEM

Pthread Scheduling API

```c
int main(int argc, char *argv[]) {
  pthread_t t1, t2;
  pthread_attr_t attr;

  pthread_attr_init(&attr);
  pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);

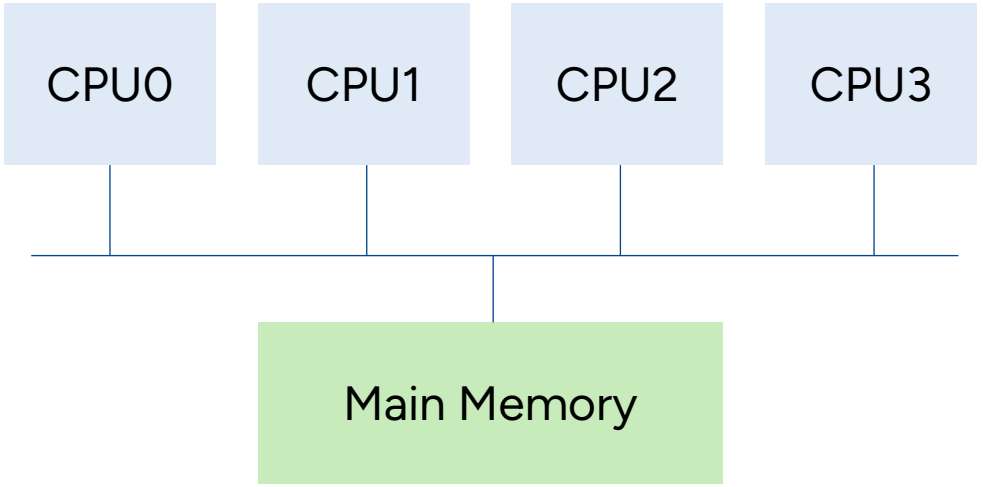
  pthread_create(&t1, &attr, thread_func, NULL);
  pthread_create(&t2, &attr, thread_func, NULL);

  pthread_join(t1, NULL);
  pthread_join(t2, NULL);
}

void *thread_func(void *param) {
  /* do some work ... */
  pthread_exit(0);
}
```

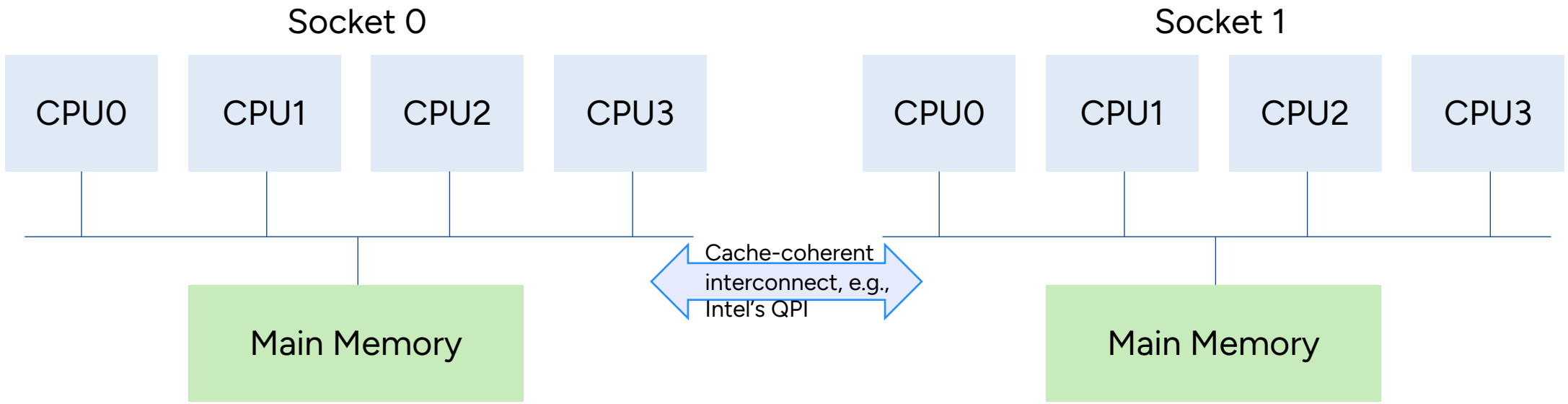
Multiprocessor

- Modern computer systems are made of multi-core processors
- More than 1 CPU to schedule tasks to.



Multiprocessor

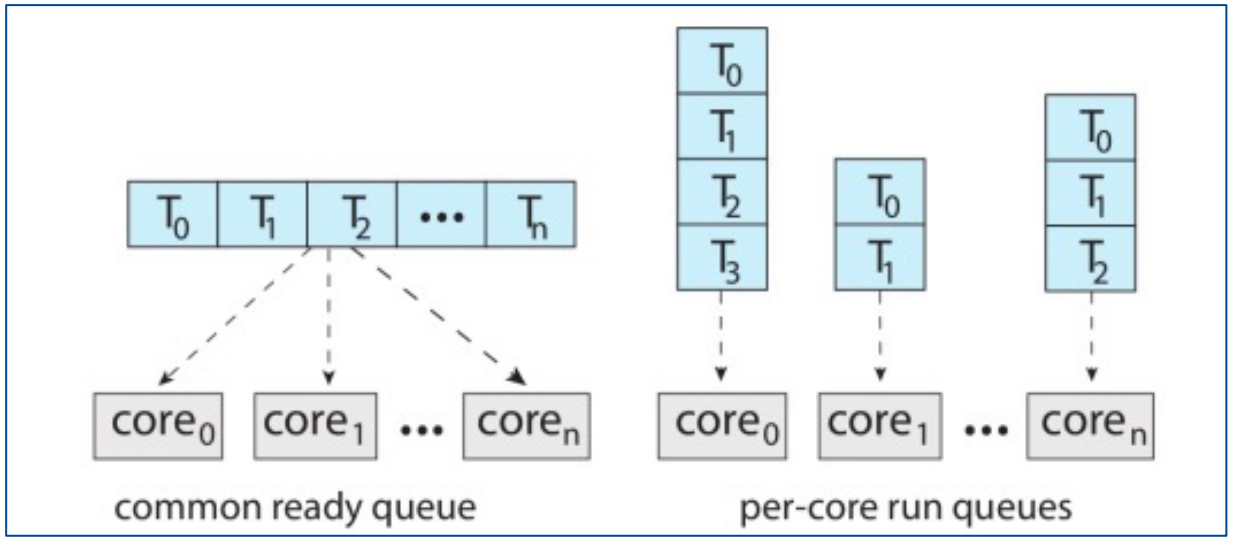
- Modern computer systems are made of multi-core processors
- More than 1 CPU to schedule tasks to.



https://en.wikipedia.org/wiki/Intel_QuickPath_Interconnect

Multiprocessor

- Modern computer systems are made of multi-core processors
- More than 1 CPU to schedule tasks to.



Optional: Apple Silicon Examples

- How many different cores does Apple Silicon have?

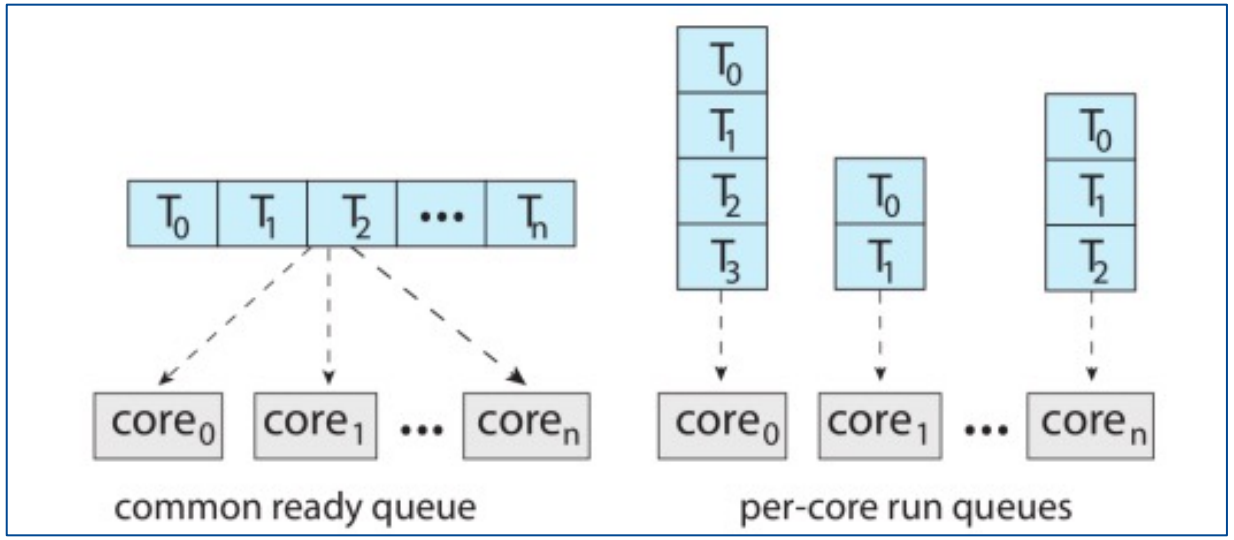
Table 1. Comparison of Baseline Apple Silicon M Series Architecture.

Feature	M1	M2	M3	M4
Process Technology (nm)	5	5/4	3	3
CPU Architecture	ARMv8.5-A	ARMv8.6-A	ARMv8.6-A	ARMv9.2-A
Performance/Efficiency Cores	4/4	4/4	4/4	4/6
Clock Frequency (GHz)	3.2 (P)/2.06 (E)	3.5 (P)/2.42 (E)	4.05 (P)/2.75 (E)	4.4 (P)/2.85 (E)
Vector Unit (name/size)	NEON/128	NEON/128	NEON/128	NEON/128
L1 Cache (KB)	128 (P)/64 (E)	128 (P)/64 (E)	128 (P)/64 (E)	128 (P)/64 (E)
L2 Cache (MB)	12 (P)/4 (E)	16 (P)/4 (E)	16 (P)/4 (E)	16 (P)/4 (E)
AMX Characteristics	FP16,32,64	FP16,32,64/BF16	FP16,32,64/BF16	FP16,32,64/BF16
GPU Cores	7-8	8-10	8-10	8-10
Native Precision Support	FP32, FP16, INT8	FP32, FP16, INT8	FP32, FP16, INT8	FP32, FP16, INT8
GPU Clock Frequency (GHz)	1.27	1.39	1.38	1.47
Theoretical FP32 FLOPS (TFLOPS)	2.29-2.61	2.86-3.57	2.82-3.53	4.26
Neural Engine Units (Core)	16	16	16	16

Hübner, Paul, Andong Hu, Ivy Peng, and Stefano Markidis. "Apple vs. Oranges: Evaluating the Apple Silicon M-Series SoCs for HPC Performance and Efficiency." IEEE IPDPSW, 2025.

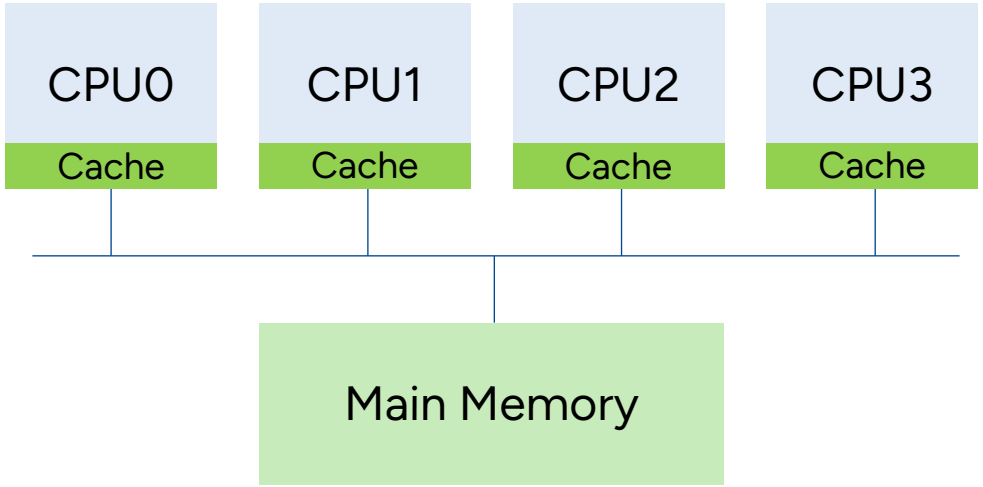
Multiple-Processor Scheduling

- Asymmetric multiprocessing
  - Only one processor does all scheduling decisions, I/O processing, and other system activities.
  - The other processors execute only user code.
- Symmetric multiprocessing (SMP)
  - Each processor is self-scheduling
  - All processes are in a common ready queue, or each processor has its own queue of ready processes.



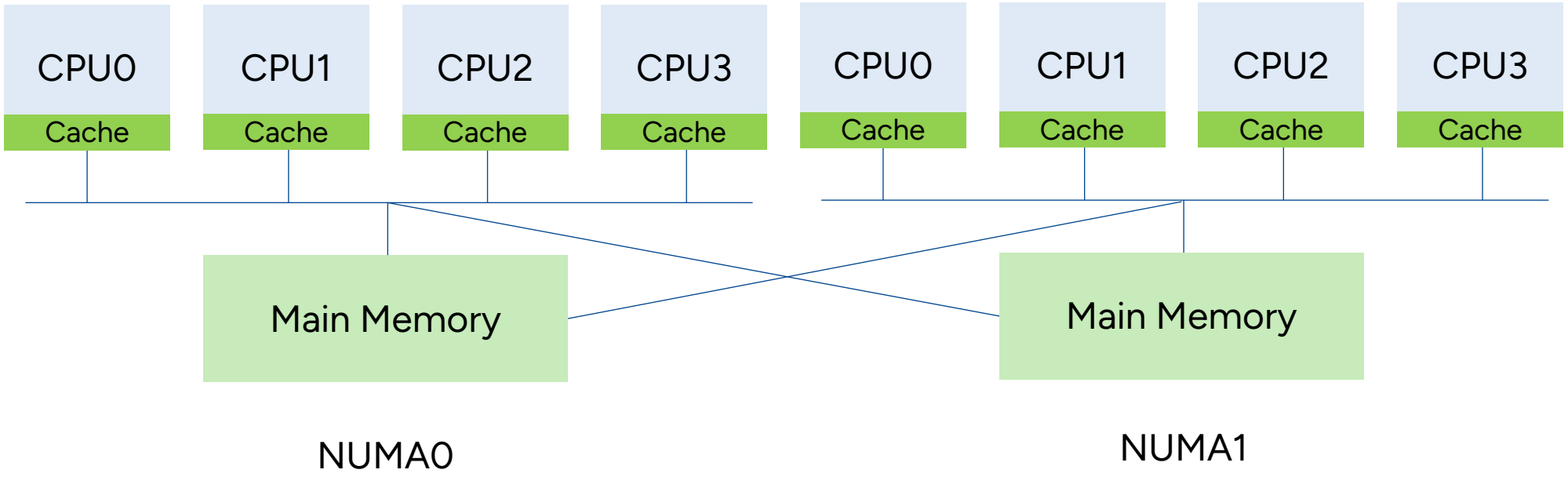
Processor Affinity

- If a thread migrate from processor to another, e.g., load Imbalance might cause thread migration
  - The contents of cache must be invalidated for the first processor
  - the cache for the second processor must be repopulated
- Most OS tries to keep a thread running on the same processor to use **warm cache**
  - **Soft affinity**: the OS attempts to keep a process on a single processor, but it is possible for a process to migrate between processors.
  - **Hard affinity**: allowing a process to specify a subset of processors on which it may run.
    - $ numactl -C 0-4 -m 0 ./app



Multi-NUMA Multiple-Processor

- NUMA: non-uniform memory access (NUMA)
  - a CPU has faster access to its local memory
  - a CPU has slow access to the remote memory on another NUMA node



CPU Affinity

- Use ‘numactl’ to check your machine’s NUMA domains
- Use `sched_setaffinity()` and `sched_getaffinity()` to set/get the CPU affinity of the process specified by `pid`.

```
ip@ip-QEMU-Virtual-Machine:/shared$ numactl -H
available: 1 nodes (0)
node 0 cpus: 0 1 2 3
node 0 size: 3899 MB
node 0 free: 1366 MB
node distances:
node   0
  0:  10
```

```
#define _GNU_SOURCE
#include <sched.h>

int sched_setaffinity(pid_t pid, size_t cpusetsize, const cpu_set_t *mask);
int sched_getaffinity(pid_t pid, size_t cpusetsize, cpu_set_t *mask);
```

Back to Our Four Questions

- Q1 The scheduling criteria, how to select a single algorithm?
  - System: CPU utilization, throughput.
  - Per-process: waiting, turnaround, response time.

- Q2 Given arrival and burst times, compute metrics under FCFS, SJF, and Round Robin.
  - Draw the Gantt chart. Then compute per-process metrics before compute system average.
  - SJF lowest average waiting; RR best responsiveness.

- Q3 Convoy effect and starvation: which algorithms suffer each, and what fixes them?
  - a long job blocks shorter ones -> convoy -> preemption / short-job ordering.
  - starvation (priority) -> aging and rotation.

- Q4 What does Linux actually run today? What changed from CFS to EEVDF?
  - CFS. EEVDF.
  - vruntime and nice value weighted fairness